Why Self-Attention Is Called “Self”

Contents

1. Overview 2
2. Prerequisites 3
3. Attention in the encoder–decoder, in three equations 3
4. Self-attention in the same three equations 5
5. Why self-attention is attention 7
6. Why it is called “self” 8
7. Summary 10
8. Sources 10
9. Key terms 11

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Self-attention (query, key, value).
- **Compare with:** Cross-attention.

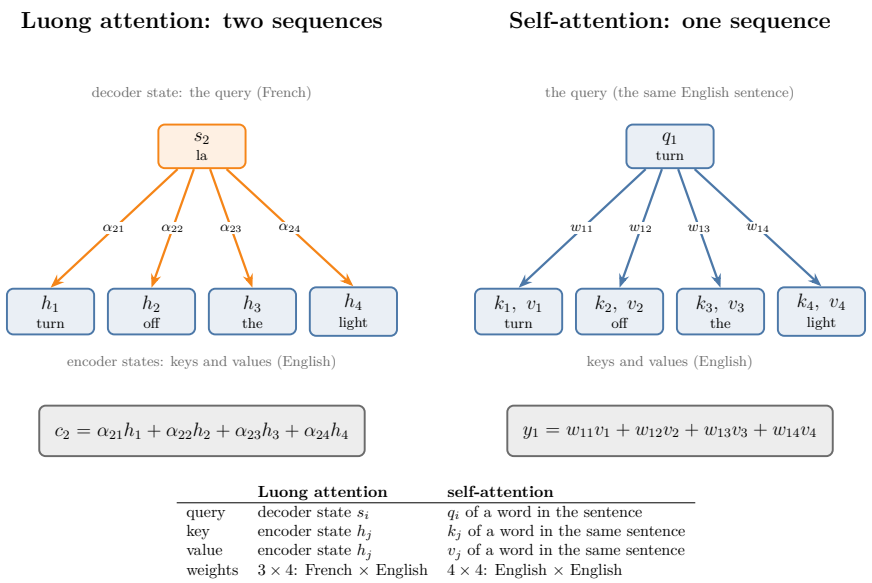
1. Overview

Key point: Self-attention (G-1763) is an attention mechanism because it uses the same three equations as Luong's attention in an encoder–decoder: dot-product scores between a query and **keys** (G-1011), a softmax, and a weighted sum of values. Self-attention is called "self" because the queries, keys and values all come from one sentence, while Luong's attention relates two different sentences, the output and the input.

At first sight self-attention looks nothing like the attention that gives the decoder a new context vector for every output word. The earlier attention lived inside an encoder–decoder with two **LSTMs** (G-1123), and it helped the decoder look back at the input sentence. Self-attention, as built in self-attention step by step, has no encoder and no decoder: it takes one sentence and returns a **contextual embedding** for every word (a vector for the word that already mixes in the other words of the sentence).

This Note answers two questions:

1. Why is self-attention a kind of **attention**?
2. Why is it called **self**?



	Luong attention	self-attention
query	decoder state s_i	q_i of a word in the sentence
key	encoder state h_j	k_j of a word in the same sentence
value	encoder state h_j	v_j of a word in the same sentence
weights	3×4: French $\times$ English	4×4: English $\times$ English

Figure 1: Left: Luong attention, where a decoder state (French) queries the encoder states (English). Right: self-attention, where a word of the English sentence queries the words of the same sentence. Bottom: the roles side by side

2. Prerequisites

- The attention mechanism: a new **context vector** (G-461) for every output word, built from all encoder states.
- Luong's simpler score: Luong's dot-product score.
- self-attention step by step: query, key and **value** (G-2068) vectors.
- **Scaled dot-product attention** (G-1745): dividing the scores by $\sqrt{d_k}$ (d_k is the length of a **key** (G-1011) vector).

3. Attention in the encoder–decoder, in three equations

> **Key point:** For output step i, the decoder state s_i is compared with every encoder state h_j by a dot product; the softmax turns the scores into weights α_{ij}; the context vector c_i is the weighted sum of the encoder states.

Recall the setting of the attention mechanism. We translate the English sentence "Turn off the light." into French, "Éteins la lumière.", a pair taken from the English–French dataset used in these Notes. An **LSTM** (G-1123) encoder reads the 4 English words and keeps a hidden state after each: h_1, h_2, h_3, h_4. An LSTM decoder writes the 3 French words, with a hidden state s_i at each step.

The plain encoder–decoder hands the decoder one context vector for the whole sentence. Attention gives the decoder a fresh context vector c_i at every step, a weighted mix of all encoder states. With the dot-product score of **Luong attention** (G-1137; Luong et al. 2015), the context vector comes from three equations:

1. **Scores:** $e_{ij} = s_i \cdot h_j$, how well encoder state j matches what the decoder needs at step i. The dot product $\cdot$ of two vectors multiplies matching entries and adds the results, so a larger **value** (G-2068) means more alike. For $[1, 2] \cdot [3, 4]$:

$$1 \times 3 + 2 \times 4$$

$$= 11$$

2. **Weights:** $\alpha_{ij} = \text{softmax}_j(e_{ij})$: the softmax over j turns the scores of one step into positive numbers that sum to 1.
3. **Context vector:** $c_i = \sum_j \alpha_{ij} h_j$, where $\sum_j$ adds the terms for $j = 1, \ldots, 4$.

For example, the **context vector** (G-461) for writing "la" (Figure 2) is:

$$c_2 = \alpha_{21} h_1 + \alpha_{22} h_2$$

$$+\alpha_{23} h_3 + \alpha_{24} h_4$$

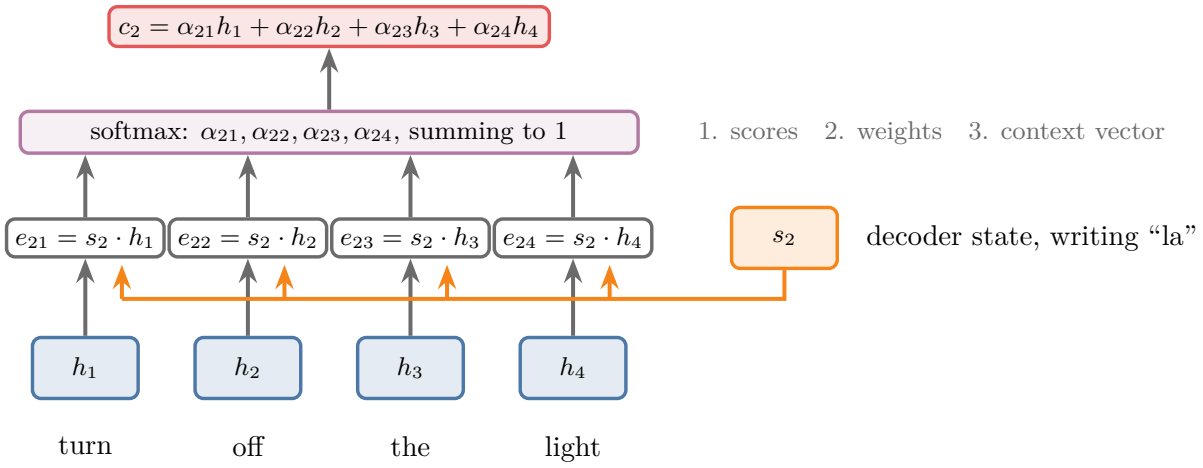


Figure 2: The three equations for decoder step 2, writing "la". The decoder state s_2 (orange) is compared with every encoder state by a dot product; the softmax turns the four scores into weights; the weights mix the encoder states into c_2

With 3 output words and 4 input words there are 12 weights α_{ij}:

$$3 \times 4 = 12$$

Luong et al. write the same equations with h_t for the decoder state and $\bar{h}_s$ for an encoder state (Luong et al. 2015, section 3: eq. 7 for the weights, eq. 8 for the score); The two compared are compared there.

4. Self-attention in the same three equations

> **Key point: Self-attention** (G-1763) follows the same three steps: query · key scores, a softmax, a weighted sum of values. Only the source of the vectors differs.

Take now only the English sentence, "turn off the light". Self-attention gives every word a query q_j, a key k_j and a value v_j, and builds the contextual embedding of "turn" like this:

1. **Scores,** for $j = 1, \ldots, 4$:

$$s_{1j} = q_1 \cdot k_j$$

2. **Weights:**

$$w_{1j} = \text{softmax}_j(s_{1j})$$

3. **Output:**

$$y_1 = w_{11}v_1 + w_{12}v_2$$

$$+w_{13}v_3 + w_{14}v_4$$

Then the same for "off", "the" and "light". Written side by side (Figure 1), the two computations match role for role:

Role	Luong attention	Self-attention
Query: the one asking "which words help me?"	decoder state s_i	query q_i of a word
Key: compared with the query	encoder state h_j	key k_j of a word
Value: mixed into the output	encoder state h_j	value v_j of a word
Similarity	$e_{ij} = s_i \cdot h_j$	$s_{ij} = q_i \cdot k_j$
Weights	$\alpha_{ij} = \text{softmax}_j(e_{ij})$	$w_{ij} = \text{softmax}_j(s_{ij})$
Output	context vector c_i	contextual embedding y_i

The decoder state acts as the **query**: at step i the decoder asks which input words will help it write the next word. The encoder states answer as **keys** (G-1011), and they are also the **values** (G-2068) that get mixed. In Luong attention the key and the value of a word are the same vector h_j; self-attention gives each word two different vectors for these two jobs.

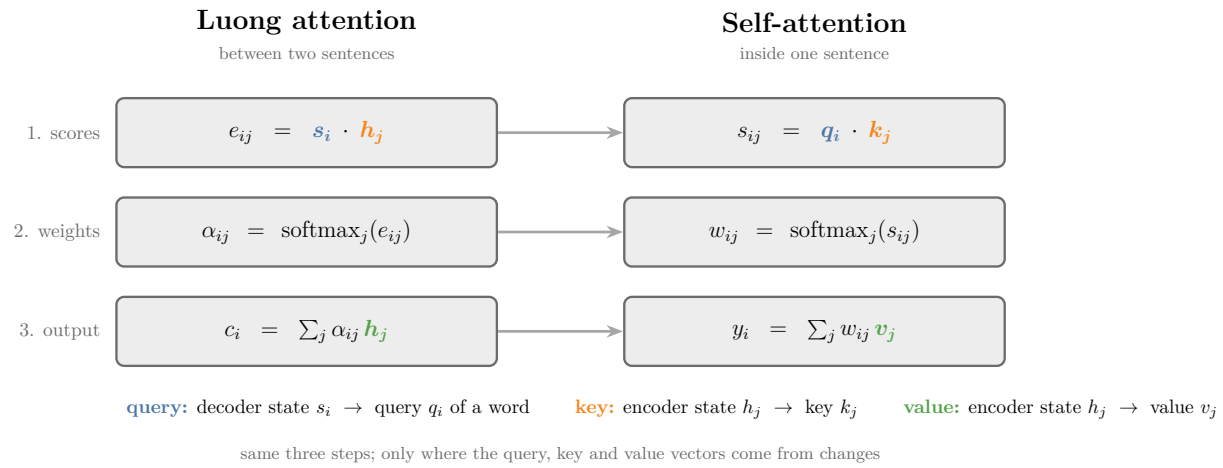


Figure 3: The three equations of Luong attention (left) and of self-attention (right), step by step. The colours mark the only substitutions: the decoder state becomes a query (blue), the encoder state used for comparing becomes a key (orange), and the encoder state that is mixed becomes a value (green)

Figure 3 lines the equations up row by row. Everything in grey is the same on both sides; only the coloured vectors are swapped.

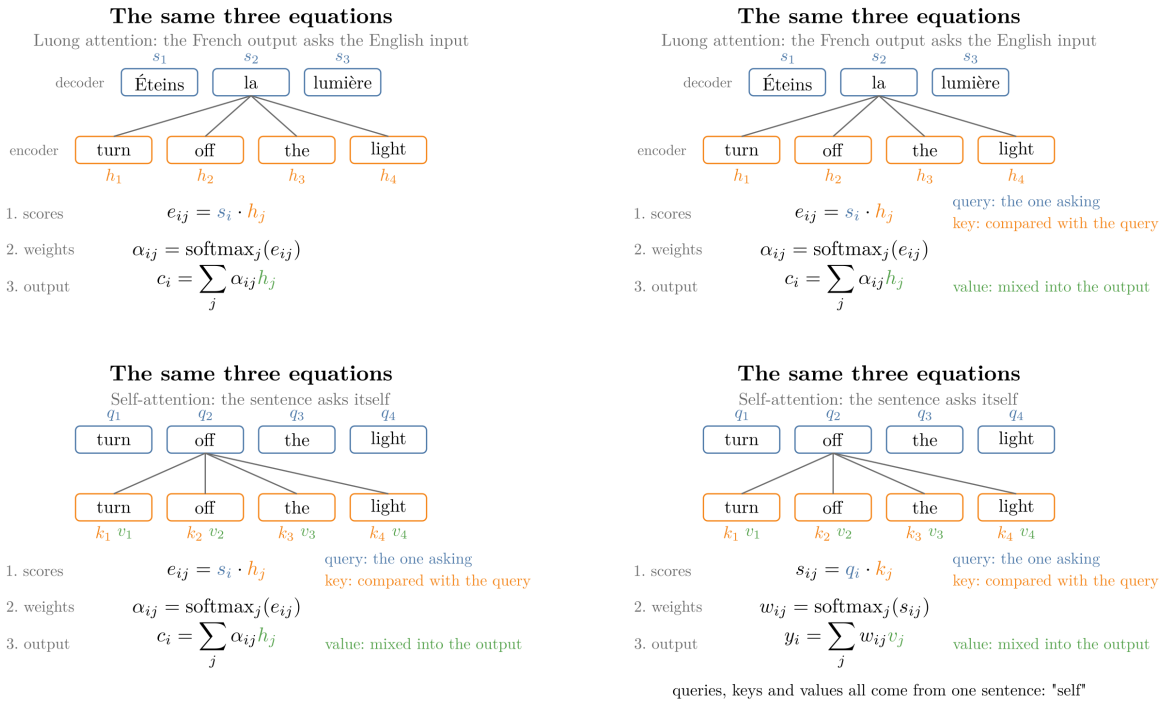


Figure 4: Luong attention turning into self-attention. The French row leaves, the English sentence takes its place and asks itself, and in each equation the decoder state becomes a query (blue), the encoder state becomes a key (orange) where it is compared and a value (green) where it is mixed

Figure 4 plays the same substitution as a movement. Watch the top first: the French words that were asking leave, and a copy of the English sentence moves up to ask instead. Then watch the equations: only the names of the vectors change, and the three steps (scores, weights, output) stay where they are.

5. Why self-attention is attention

> **Key point:** The mathematics is the same. One function, and one Keras layer, computes both.

Since the three equations are the same, one function computes both kinds of attention (Figure 5). The Notebook writes it once:

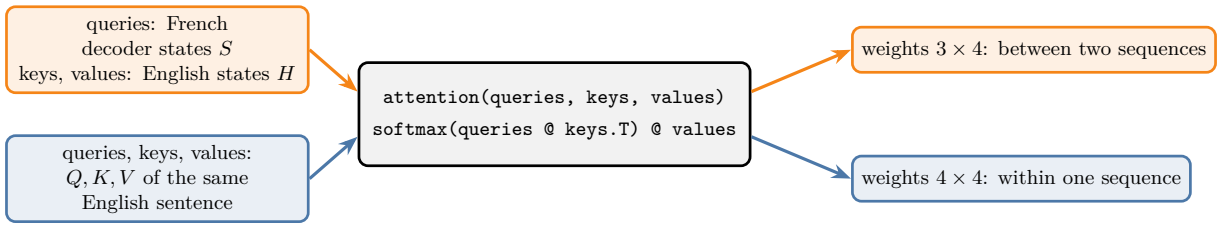


Figure 5: One function, two calls. Fed the French decoder states as queries and the English states as keys and values, it relates two sequences; fed the queries, keys and values of one sentence, it relates the sentence to itself

Python: The three equations as one function.

```
def attention(queries, keys, values):
    A = softmax(queries @ keys.T)      # scores, then weights
    return A, A @ values               # weights, weighted sums

A_luong, C = attention(S, H, H)        # decoder states ask the encoder
↪ states
A_self, Y = attention(Q / np.sqrt(d), K, V)   # one sentence asks
↪ itself
```

The Notebook runs it on the sentence pair above, with an (untrained) **LSTM** (G-1123) encoder and decoder of 8 units:

- **Between two sequences:** `attention(S, H, H)` gives a 3×4 weight matrix, French words by English words. Keras' `keras.layers.Attention` (G-103), which its documentation calls "dot-product attention layer, a.k.a. Luong-style attention", returns exactly the same weights and **context vectors** (G-461).
- **Within one sequence:** the input is the matrix X of English embeddings, one row per word. Three weight matrices W_Q, W_K, W_V turn each embedding into a query, a key and a **value** (G-2068):

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

With the scores divided by $\sqrt{d_k}$, the same function gives a 4 by 4 weight matrix, English by English: 4 words, each scoring all 4 words, so 16 weights, and each row adds up to 1. The same Keras layer again returns the same numbers.

Self-attention is attention because it is the same computation: a query scores **keys** (G-1011), the softmax turns the scores into weights, and the weights mix the values. Only the setting, an encoder–decoder or a single sentence, made the two look different.

6. Why it is called "self"

> **Key point:** Luong and Bahdanau attention work between two sequences (**inter-sequence attention**, G-958). **Self-attention** (G-1763) works within one sequence (intra-sequence): the sequence attends to itself.

In Luong's attention the queries come from one sequence, the French output, and the **keys** (G-1011) and values from another, the English input. The weights relate the words of two different sentences, so the weight matrix is rectangular: 3×4, output length by input length.

In self-attention the queries, keys and **values** (G-2068) all come from the same sentence. Every word is compared with every word of its own sentence, itself included, and the weight matrix is square: 4×4. Vaswani et al. (2017, section 2) define it this way: "Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence."

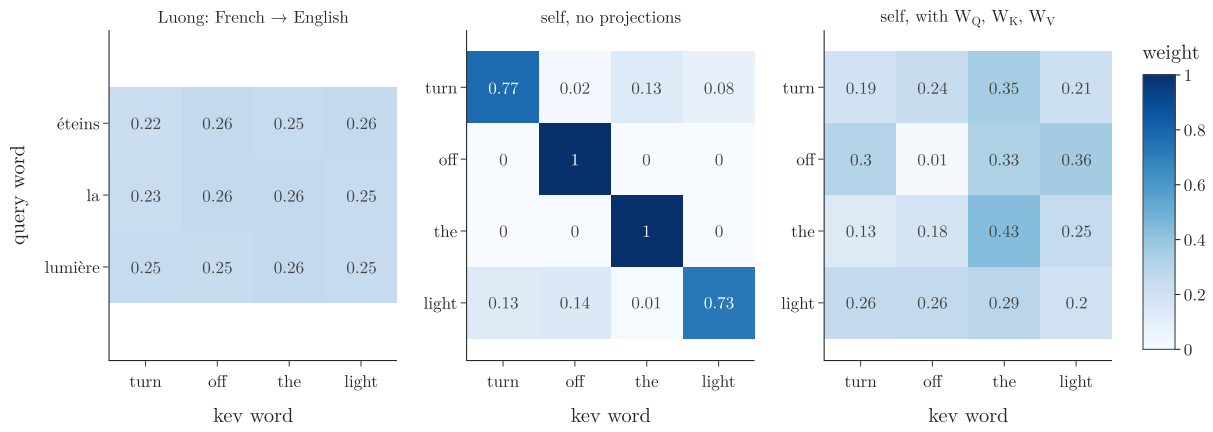


Figure 6: Weight matrices from the Notebook (untrained networks, so the values have no meaning yet; the shapes do). Left: Luong attention, French queries over English keys, 3×4. Middle: self-attention on the English embeddings without projections. Right: with W_Q, W_K, W_V and scaling. Both self-attention matrices are 4×4

The "self" therefore describes where the inputs come from, not a different calculation. Keras' `MultiHeadAttention` layer shows this directly: it takes a `query` sequence and a `value` sequence. Called with the English sentence as both, it does self-attention and returns 4×4 weights; called with the French states as the query and the English sentence as the value, the same layer returns 3×4 weights (Notebook). The second use, between the decoder and the encoder of a transformer, is called **cross-attention** (G-507) and has its own section in **cross-attention** (G-507).

> **Extra:** The middle panel of Figure 6 shows why self-attention needs its projection matrices. Without W_Q and W_K, the query and the key of a word are the same vector, and the dot product of a vector with itself (its squared length) is usually the largest score: "off" and "the" put all their weight on themselves. Jurafsky and Martin (SLP3 draft, ch. 7, section on attention) note the same: in the simple version, "the softmax weight will likely be highest for x_i, since x_i is very similar to itself". Separate query, key and value matrices, (see self-attention step by step), let a word look for something other than itself (right panel).

7. Summary

	Luong attention	Self-attention
Sequences	two: output and input	one
Query	decoder state s_i	$q_i = x_i W_Q$
Keys and values	encoder states h_j (same vector for both)	$k_j = x_j W_K$, $v_j = x_j W_V$
Scaling	none	divide by $\sqrt{d_k}$
Weight matrix	output length × input length (3×4)	length × length (4×4)
Output	context vector for the decoder	contextual embedding of every word

- Self-attention is attention: scores from query · **key** (G-1011) dot products, a softmax, and a weighted sum of values, the same three equations as Luong's attention.
- It is "self" because one sequence supplies the queries, the keys and the **values** (G-2068): attention within a sequence (intra-attention), not between two.
- **Self-attention** (G-1763) gives each word its own query, key and value through W_Q, W_K, W_V, because without them a word's query equals its key and most of its weight goes to itself.
- One function, or one Keras layer, computes both; only the inputs differ, so the same layer later serves as self-attention and as **cross-attention** (G-507) in the transformer.

8. Sources

Built from

- CampusX, "Why is Self Attention called"Self"? | Self Attention Vs Luong Attention in Depth Lecture | CampusX", YouTube, https://www.youtube.com/watch?v=o4ZVA0TuDRg

Other references

- Luong, M.-T., Pham, H. and Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *EMNLP 2015*. arXiv:1508.04025. Section 3, eq. 7 and 8 (the dot score, the weights $a_t(s)$, the context vector c_t).
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Section 2 (self-attention, "sometimes called intra-attention"); section 3.2.1 (scaled dot-product attention).
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026). Chapter 14, section 14.8 (dot-product attention in the encoder–decoder); chapter 7, section on attention (the simple version weighs x_i most).
- Tatoeba Project, English–French sentence pairs, distributed by manythings.org/anki (the Keras `fra-eng` example dataset).
- Keras API documentation: `Attention` layer ("Dot-product attention layer, a.k.a. Luong-style attention") and `MultiHeadAttention` layer, keras.io/api/layers/attention_layers.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Self-attention (intra-attention) (G-1763)	Attention in which the words of one sequence attend to each other (the queries, keys and values all come from that sequence); it gives each word a new vector that depends on the words around it.
`keras.layers.Attention` (G-103)	Keras' layer that lets each item of one sequence look at another sequence: it scores queries from the first against keys from the second by dot product (Luong-style attention). It returns the attention weights and the weighted sums of the values (context vectors).
Inter-sequence attention (G-958)	Attention between two different sequences, such as an output and an input sentence.
Key (G-1011)	In attention, the vector each word offers to be compared with a query; the dot product of the query with a key scores how much attention that word gets.
Long short-term memory (LSTM) (G-1123)	A recurrent network (RNN) that carries two memories from one time step to the next, a long-term one (the cell state) and a short-term one (the hidden state), with gates that control what each memory keeps and passes on.
Context vector (G-461)	The summary of the input that the decoder works from; one fixed vector in the plain encoder–decoder, a new one per output word with attention.
Value (G-2068)	In self-attention, the vector each word contributes to the weighted sum that becomes a word's new vector; it comes from the word's embedding through a learned value matrix.
Scaled dot-product attention (G-1745)	Self-attention in which every query–key dot-product score is divided by $\sqrt{d_k}$ before the softmax, so the scores do not spread so wide that the softmax saturates: $\text{softmax}(QK^T/\sqrt{d_k})\,V$.

Term	Meaning
Luong attention (multiplicative, dot) (G-1137)	Attention between a decoder state and the encoder states, scored by multiplying them: $s_i^\top h_j$ (dot) or $s_i^\top W_a h_j$.
Cross-attention (G-507)	Attention between the two halves of a transformer: the queries come from the decoder and the keys and values from the encoder's output, so each output word can use the input sentence.
Query, key, value vectors (G-1607)	Three vectors made from each word's embedding by learned matrices: a word's query is compared with every word's key by dot products to get attention weights, and those weights mix the value vectors into the word's new vector.